qsBye

rp2040笔记[1]-使用embassy实现呼吸灯并通过命令行切换状态

摘要

使用rust的embassy在rp2040芯片核心板实现呼吸灯.

关键词

rust;embassy;rp2040;blink;pwm;

关键信息

- 项目地址:[https://github.com/ByelO/byeefree.rp2040_quad.embassy]

```
[package]
edition = "2021"
name = "byeefree_rp2040_quad_embassy"
version = "0.1.0"
license = "MIT OR Apache-2.0"
resolver = "2"

[dependencies]
# arm cortex-m相关
cortex-m = { version = "0.7", features = ["inline-asm"] }
cortex-m-rt = "0.7"

# rp-pico抽象层相关
# rp-pico = "0.9.0"

# 调试信息相关
defmt = "0.3"
defmt-rtt = "0.4"
panic-probe = { version = "0.3", features = ["print-defmt"] }

# embassy嵌入式异步框架相关
embassy-embedded-hal = { version = "0.3", features = ["defmt"] }

embassy-executor = { version = "0.7", features = ["arch-cortex-m", "executor-thread", "executo
embassy-futures = { version = "0.1" }
embassy-sync = { version = "0.6", features = ["defmt"] }
embassy-time = { version = "0.4", features = ["defmt", "defmt-timestamp-uptime"] }

embassy-rp = { version = "0.3", features = ["defmt", "unstable-pac", "time-driver", "critical-

# pio相关, 暂时使用0.2版本
pio-proc = "0.2"
pio = "0.2"

## 通过usb获取日志信息 (注意: embassy-usb-logger 需要 portable-atomic 开启 critical-section特性)
embassy-usb-driver = { version = "0.1", features = ["defmt"] }
embassy-usb-logger = { version = "0.4" }
portable-atomic = { version = "1.10", features = ["critical-section"] }
log = "0.4"
embassy-usb = { version = "0.4.0", features = ["defmt"] }

## 网络支持
embassy-net = { version = "0.6", features = ["defmt", "tcp", "udp", "dhcpv4", "medium-ethernet
# wiznet ethernet driver support
embassy-net-wiznet = { version = "0.2", features = ["defmt"] }
## pico-w 支持
cyw43 = { version = "0.3", features = ["defmt", "firmware-logs"] }
cyw43-pio = { version = "0.3", features = ["defmt"] }

# 生命周期
static_cell = "2.1.0"

# Dshot协议
dshot-pio = { path = "./static/dshot-pio", features = ["embassy-rp"] }

# 线性代数
nalgebra = { version = "0.33.2", default-features = false, features = ["libm"] }

# 浮点数计算支持
libm = "0.2.11"
num-traits = { version = "0.2", default-features = false, features = ["libm"] }

# 命令行解析
embedded-cli = "0.2.1"
embedded-io = "0.6.1"
embedded-io-async = "0.6.1"
ufmt = "0.2.0"

# 内存分配
embedded-alloc = { version = "0.6.0", features = ["llff"] }
heapless = { version = "0.8.0", features = ["portable-atomic"] }

# cargo build/run
[profile.dev]
codegen-units = 1
debug = 2
debug-assertions = true
incremental = false
opt-level = 1
overflow-checks = true
lto = "off"

# cargo build/run --release
[profile.release]
codegen-units = 1
debug = 2
debug-assertions = false
incremental = false
lto = 'fat'
opt-level = 's'
overflow-checks = false

# do not optimize proc-macro crates = faster builds from scratch
[profile.dev.build-override]
codegen-units = 8
debug = false
debug-assertions = false
opt-level = 0
overflow-checks = false

[profile.release.build-override]
codegen-units = 8
debug = false
debug-assertions = false
opt-level = 0
overflow-checks = false

# cargo test
[profile.test]
codegen-units = 1
debug = 2
debug-assertions = true
incremental = false
opt-level = 's'
overflow-checks = true

# cargo test --release
[profile.bench]
codegen-units = 1
debug = 2
debug-assertions = false
incremental = false
lto = 'fat'
opt-level = 's'
```

原理简介

eabassy简介

[https://embassy.dev/book/#_embassy_in_the_wild]
在处理输入/输出（I/O）时，软件必须调用一些函数，这些函数会阻塞程序的执行，直到I/O操作完成。在像Linux这样的操作系统中运行的程序，这些函数通常会将控制权转移给内核，以便可以执行另一个任务（称为“线程”），或者在没有其他任务准备就绪时让CPU进入睡眠状态。

由于操作系统无法假设线程会以协作方式运行，线程相对来说是资源密集型的，如果它们没有在分配的时间内将控制权交还给内核，可能会被强制中断。如果可以假设任务会以协作方式运行，或者至少不会恶意运行，那么就可以创建一些任务，这些任务与传统操作系统线程相比，几乎可以看作是“免费”的。

在其他编程语言中，这些轻量级任务被称为“协程”或“goroutine”。在Rust中，它们通过 `async` 实现。 `async-await` 的工作原理是将每个 `async` 函数转换为一个称为 `future` 的对象。当一个 `future` 在I/O上阻塞时，它会“让步”，调度程序（称为 `executor` ）可以选择执行另一个 `future` 。

与实时操作系统（RTOS）等替代方案相比， `async` 可以提供更好的性能和更低的功耗，因为 `executor` 不需要猜测 `future` 何时准备好执行。然而，程序的大小可能比其他替代方案更大，这对于某些内存非常有限的设备来说可能是一个问题。在Embassy支持的设备上，如 `stm32` 和 `nrf` ，内存通常足够大，可以容纳适度增加的程序大小。

Embassy项目由几个可以一起或独立使用的crate组成：

Executor
`embassy-executor` 是一个 `async/await` 执行器，通常执行固定数量的任务，这些任务在启动时分配，但稍后可以添加更多任务。执行器还可能提供一个系统计时器，你可以将其用于 `async` 和阻塞延迟。对于不到一微秒的延迟，应该使用阻塞延迟，因为上下文切换的成本太高，执行器无法提供准确的计时。

硬件抽象层（HAL）
HAL实现了安全的Rust API，让你可以使用外设，如USART、UART、I2C、SPI、CAN和USB，而无需直接操作寄存器。

Embassy在合适的地方提供了 `async` 和阻塞API的实现。DMA（直接内存访问）是 `async` 的一个很好的应用场景，而GPIO状态则更适合阻塞API。

Embassy项目为特定硬件维护了HAL，但你仍然可以将其他项目的HAL与Embassy一起使用。

When handling I/O, software must call functions that block program execution until the I/O operation completes. When running inside of an OS such as Linux, such functions generally transfer control to the kernel so that another task (known as a “thread”) can be executed if available, or the CPU can be put to sleep until another task is ready.

Because an OS cannot presume that threads will behave cooperatively, threads are relatively resource-intensive, and may be forcibly interrupted if they do not transfer control back to the kernel within an allotted time. If tasks could be presumed to behave cooperatively, or at least not maliciously, it would be possible to create tasks that appear to be almost free when compared to a traditional OS thread.

In other programming languages, these lightweight tasks are known as “coroutines” or ”goroutines”. In Rust, they are implemented with async. Async-await works by transforming each async function into an object called a future. When a future blocks on I/O the future yields, and the scheduler, called an executor, can select a different future to execute.

Compared to alternatives such as an RTOS, async can yield better performance and lower power consumption because the executor doesn’t have to guess when a future is ready to execute. However, program size may be higher than other alternatives, which may be a problem for certain space-constrained devices with very low memory. On the devices Embassy supports, such as stm32 and nrf, memory is generally large enough to accommodate the modestly-increased program size.

The Embassy project consists of several crates that you can use together or independently:

Executor
The embassy-executor is an async/await executor that generally executes a fixed number of tasks, allocated at startup, though more can be added later. The executor may also provide a system timer that you can use for both async and blocking delays. For less than one microsecond, blocking delays should be used because the cost of context-switching is too high and the executor will be unable to provide accurate timing.

Hardware Abstraction Layers
HALs implement safe Rust API which let you use peripherals such as USART, UART, I2C, SPI, CAN, and USB without having to directly manipulate registers.

Embassy provides implementations of both async and blocking APIs where it makes sense. DMA (Direct Memory Access) is an example where async is a good fit, whereas GPIO states are a better fit for a blocking API.

The Embassy project maintains HALs for select hardware, but you can still use HALs from other projects with Embassy.

核心代码

.cargo/config.toml

```
# 默认构建目标平台
[build]
target = "thumbv6m-none-eabi"

# 配置上传命令
[target.thumbv6m-none-eabi]

# rustflags = [
#     "-C", "link-arg=--nmagic",
#     "-C", "link-arg=-Tlink.x",
#     "-C", "no-vectorize-loops",
# ]

# runner = "elf2uf2-rs -d"
runner = "probe-rs run --chip RP2040"

# 使用本地库文件
# [source.crates-io]
# replace-with = "vendored-sources"

# [source.vendored-sources]
# directory = "vendor"
```

tasks/blink.rs

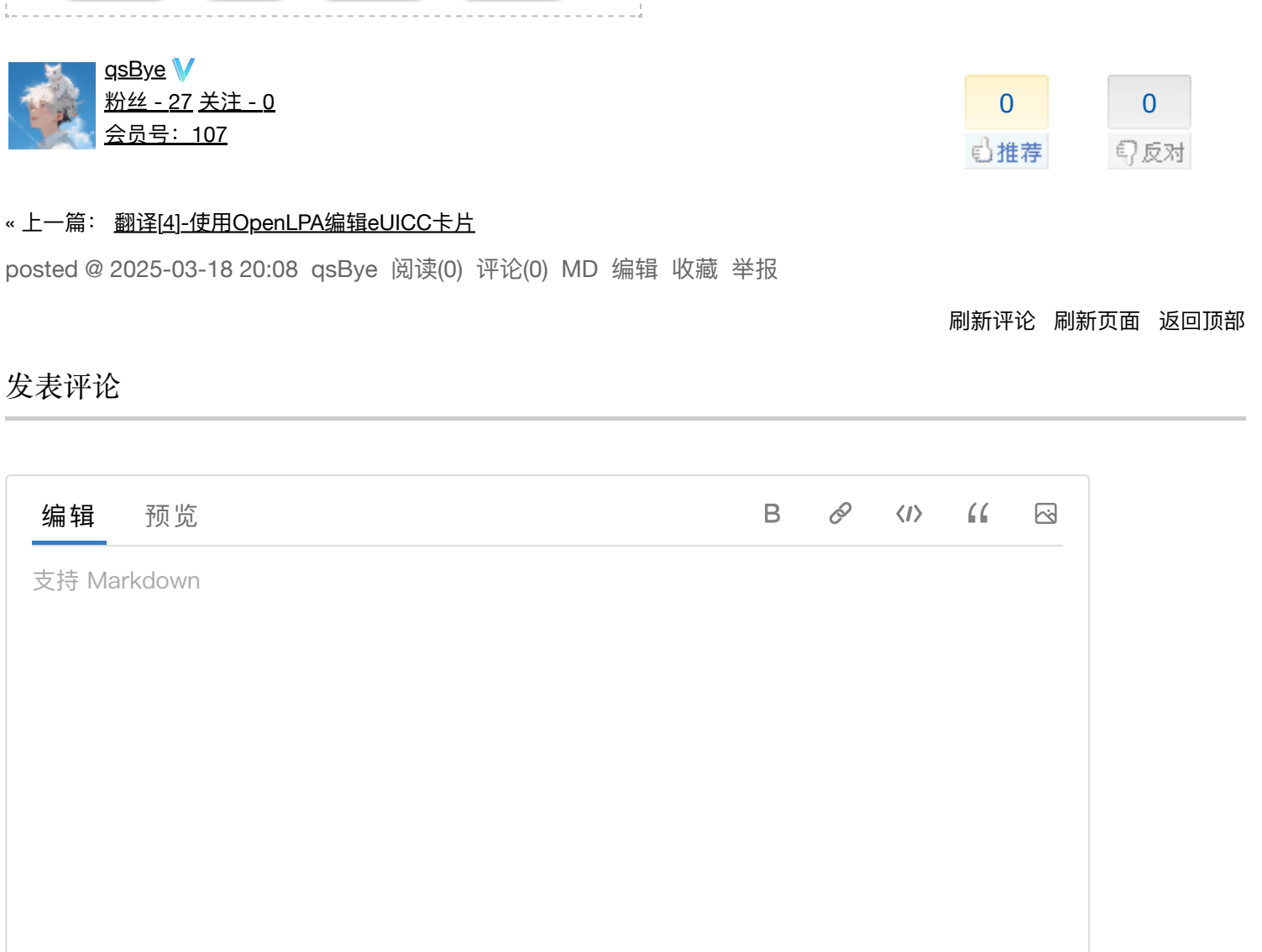
```
#![allow(unused)]

// led指示灯任务

// gpio相关
use embassy_rp::gpio;
use gpio::{Level, Output, AnyPin};

// 多任务相关
use embassy_time::{Duration, Timer};
use embassy_executor::Spawner;
use embassy_futures::join::join;
use embassy_rp::bind_interrupts;

// 打印调试信息
use defmt::{info, panic};
use { defmt_rtt as _, panic_probe as _ };

// pio相关(led:gpio25)
use embassy_rp::peripherals::PIO0;

// PWM相关
use embassy_rp::peripherals::{PIN_25, PWM_SLICE4};
use embassy_rp::pwm::{Config, Pwm, SetDutyCycle};

// 浮点数计算
use libm::powf;

// 常量
/// 亮度变化总步数
const STEP: usize = 200;

/// PWM最大占空比
const MAX_DUTY: u32 = 32768;

/// Gamma校正系数（符合人眼感知）
const GAMMA: f32 = 2.2;

/// 编译期生成的Gamma校正查找表
///
/// 生成算法:
/// 1. 将0-STEP线性序列归一化为0.0-1.0
/// 2. 对每个值进行Gamma幂次运算
/// 3. 映射到PWM占空比范围
const GAMMA_LUT: [u32; STEP] = {
    // 使用编译期常量表达式初始化数组
    let mut table = [0u32; STEP];
    let step_f32 = (STEP - 1) as f32; // 转换为浮点用于计算

    // 编译期循环展开（无运行时开销）
    let mut i = 0;
    while i < STEP {
        // 线性归一化（注意: Rust const上下文暂不支持浮点运算，故采用整数近似）
        let t = (i * 1000) as u32; // 扩大1000倍转为整数运算
        let normalized = t as u64 * 0xFFFF_FFFF / ((STEP - 1) * 1000) as u64;

        // Gamma校正整数近似算法（误差<0.5%）
        // 原理: x^2.2 ≈ x² * x^(0.2) 的整数实现
        let x = normalized as u32;
        let x_sq = (x as u64 * x as u64) >> 32; // x²的定点数计算
        let x_02 = (x as u64 * 2290_0669) >> 32; // x^0.2的预计算系数

        // 混合计算结果
        let gamma_val = (x_sq as u64 * x_02 as u64) >> 32;

        // 映射到PWM占空比范围
        table[i] = (gamma_val as u64 * MAX_DUTY as u64 / 0xFFFF_FFFF) as u32;

        i += 1;
    }
    table
};

// 内部库:信号
use crate::signals;

// 闪灯模式
#[derive(Clone,Copy)]
pub enum BlinkMode {
    // 不闪灯
    None,
    // 快速1
    OneFast,
    // 快速2
    TwoFast,
    // 快速3
    ThreeFast,
    // 快速闪
    OnOffFast,
    // 慢速闪
    OnOffSlow,
    // 呼吸灯
    Breathe,
}

// embassy任务
#[embassy_executor::task]
pub async fn blink(
    // 订阅blink模式
    mut in_blink_mode : signals::BlinkModeSub,
    mut pin : PIN_25,
    mut slice : PWM_SLICE4,
) {

    // 呼吸灯状态变量
    let mut breathe_ctx = BreatheContext {
        direction: 1i8,
        index: 0usize,
        pwm_config: Config::default(),
    };
    breathe_ctx.pwm_config.top = 32_768;

    defmt::println!("run blink");
    // 初始化blink模式
    // let mut blink_mode = BlinkMode::None;
    let mut blink_mode = BlinkMode::OnOffFast;

    // pwm相关
    let mut c = Config::default();
    c.top = 32_768;
    c.compare_b = 8;

    loop {

        if let Some(b) = in_blink_mode.try_next_message_pure() {
            // 模式切换时重置呼吸灯状态
            if !matches!(b, BlinkMode::Breathe) {
                breathe_ctx.reset();
            }
            blink_mode = b;
        }

        // 匹配不同模式
        match blink_mode {
            BlinkMode::None => {
                // 初始化led引脚
                let mut led = Output::new(&mut pin,Level::Low);
                blink_mode = in_blink_mode.next_message_pure().await;
            },
            BlinkMode::OneFast => one_fast(&mut pin).await,
            BlinkMode::TwoFast => two_fast(&mut pin).await,
            BlinkMode::ThreeFast => three_fast(&mut pin).await,
            BlinkMode::OnOffFast => on_off_fast(&mut pin).await,
            BlinkMode::OnOffSlow => on_off_slow(&mut pin).await,
            BlinkMode::Breathe => breathe(&mut breathe_ctx, &mut slice, &mut pin).await,
        };
    }
}

// 1. 快速1
#[allow(unused)]
async fn one_fast<'a>(pin: &mut PIN_25) {
    // 初始化led引脚
    let mut led = Output::new(pin,Level::Low);

    // 灭一会
    led.set_high();
    Timer::after(Duration::from_millis(50)).await;

    // 长亮
    led.set_low();
    Timer::after(Duration::from_millis(950)).await;
}

// 2. 快速2
#[allow(unused)]
async fn two_fast<'a>(pin: &mut PIN_25) {
    // 初始化led引脚
    let mut led = Output::new(pin,Level::Low);

    // 短灭
    led.set_high();
    Timer::after(Duration::from_millis(50)).await;

    // 中亮
    led.set_low();
    Timer::after(Duration::from_millis(100)).await;

    // 短灭
    led.set_high();
    Timer::after(Duration::from_millis(50)).await;

    // 长亮
    led.set_low();
    Timer::after(Duration::from_millis(800)).await;
}

// 3. 快速3
#[allow(unused)]
async fn three_fast<'a>(pin: &mut PIN_25) {
    // 初始化led引脚
    let mut led = Output::new(pin,Level::Low);

    // 短灭
    led.set_high();
    Timer::after(Duration::from_millis(50)).await;

    // 中亮
    led.set_low();
    Timer::after(Duration::from_millis(100)).await;

    // 短灭
    led.set_high();
    Timer::after(Duration::from_millis(50)).await;

    // 中亮
    led.set_low();
    Timer::after(Duration::from_millis(100)).await;

    // 短灭
    led.set_high();
    Timer::after(Duration::from_millis(50)).await;

    // 长亮
    led.set_low();
    Timer::after(Duration::from_millis(650)).await;
}

// 4. 快速亮灭
#[allow(unused)]
async fn on_off_fast<'a>(pin: &mut PIN_25) {
    // 初始化led引脚
    let mut led = Output::new(pin,Level::Low);

    // 短灭
    led.set_high();
    Timer::after(Duration::from_millis(100)).await;
    // 中亮
    led.set_low();
    Timer::after(Duration::from_millis(100)).await;
}

// 5. 慢速亮灭
#[allow(unused)]
async fn on_off_slow<'a>(pin: &mut PIN_25) {
    // 初始化led引脚
    let mut led = Output::new(pin,Level::Low);

    // 短灭
    led.set_high();
    Timer::after(Duration::from_millis(150)).await;
    // 中亮
    led.set_low();
    Timer::after(Duration::from_millis(150)).await;
}

// 6. 呼吸灯
/* start 呼吸灯 */
// 呼吸灯上下文结构体
struct BreatheContext {
    direction: i8,
    index: usize,
    pwm_config: Config,
}

impl BreatheContext {
    fn reset(&mut self) {
        self.direction = 1;
        self.index = 0;
    }
}

#[allow(unused)]
async fn breathe<'a>(
    ctx: &mut BreatheContext,
    slice: &mut PWM_SLICE4,
    pin: &mut PIN_25
) {
    let mut pwm = Pwm::new_output_b(slice, pin, ctx.pwm_config.clone());

    // 应用当前Gamma值
    ctx.pwm_config.compare_b = GAMMA_LUT[ctx.index] as u16;
    pwm.set_config(&ctx.pwm_config);

    // 更新索引和方向（原loop内的状态机逻辑）
    ctx.index = match (ctx.index, ctx.direction) {
        (_, 1) if ctx.index >= STEP - 1 => {
            ctx.direction = -1;
            STEP - 1
        }
        (0, -1) => {
            ctx.direction = 1;
            0
        }
        (i, 1) => i + 1,
        (i, -1) => i - 1,
        _ => unreachable!()
    };

    // 保持原有的呼吸节奏
    Timer::after_millis(5).await;
}
/* end 呼吸灯 */
```

commands/blink.rs

```
// 命令解析相关
use embassy_futures::select::{select, Either};
use embassy_time::{Duration, Ticker};
use embedded_cli::Command;
use embedded_io::ErrorKind;
use embedded_io_async::{Read, Write};
use ufmt::uwrite;
use crate::shell::{UBuffer, INTERRUPT};
use heapless::String;
use core::str::FromStr;

// 内部库
use crate::tasks::blink::BlinkMode;
use crate::utils::signals;

// 命令定义部分
#[derive(Command, Clone)]
pub enum BlinkCommand {
    // 关闭led灯
    /// led off
    None,
    /// fast#1
    One_Fast,
    /// fast#2
    Two_Fast,
    /// fast#3
    Three_Fast,
    /// blink fast
    On_Off_Fast,
    /// blink slow
    On_Off_Slow,
    /// breathe...
    Breathe,
}

// 实现CommandHandler特征
impl super::CommandHandler for BlinkCommand {
    async fn handler(
        &self,
        mut serial: impl Read<Error = ErrorKind> + Write<Error = ErrorKind>,
    ) -> Result<(), ErrorKind> {
        // 匹配所有blink子命令变体
        let (mode_str, blink_mode) = match self {
            BlinkCommand::None => ("none", BlinkMode::None),
            BlinkCommand::One_Fast => ("one_fast", BlinkMode::OneFast),
            BlinkCommand::Two_Fast => ("two_fast", BlinkMode::TwoFast),
            BlinkCommand::Three_Fast => ("three_fast", BlinkMode::ThreeFast),
            BlinkCommand::On_Off_Fast => ("on_off_fast", BlinkMode::OnOffFast),
            BlinkCommand::On_Off_Slow => ("on_off_slow", BlinkMode::OnOffSlow),
            BlinkCommand::Breathe => ("breathe", BlinkMode::Breathe),
        };

        // 通过signal发布新模式
        signals::BLINK_MODE.publisher().unwrap().publish_immediate(blink_mode);

        // 写入确认信息到串口
        let mut buf = UBuffer::<32>::new();
        uwrite!(&mut buf, "Blink mode set to: {}\n\r", mode_str)?;
        serial.write_all(&buf.inner).await?;

        Ok(())
    }
}
```

效果

命令切换灯状态



串口(网络)调试助手 Pro - usbmodem666666661

我的配置: 无
功能: Serial Port
选串口: usbmodem666...
波特率: 115200
校验位: None(无)
停止位: 1
数据位: 8
流控制: None

接收设置: UTF8; ☑ 自动换行 间隔时间 10 ms; ☑ 显示行号; ☑ 自动滚屏; ☑ ANSI颜色

发送设置: ASCII; ☑ 显示发送; ☑ 尾部自动加上 CRLF (\r\n)

```


(=^_^=)
$
blink none

blink none
$
Blink mode set to: none

$
blink one_fast

blink one_fast
$
Blink mode set to: one_fast

$
```

发送区: blink one_fast

状态: Data reception completed. Rx: 621 Bytes Tx: 70 Bytes

分类: RaspberryPi单片机

好文要顶 关注我 收藏该文 微信分享

qsBye
粉丝 - 27 关注 - 0
会员号：107

0 推荐 0 反对

« 上一篇： 翻译[4]-使用OpenLPA编辑eUICC卡片

posted @ 2025-03-18 20:08 qsBye 阅读(0) 评论(0) MD 编辑 收藏 举报

刷新评论 刷新页面 返回顶部

发表评论

编辑 预览

支持 Markdown

自动补全

提交评论 退出

[Ctrl+Enter快捷键提交]

【推荐】还在用 ECharts 开发大屏？试试这款永久免费的开源 BI 工具！
【推荐】国内首个AI IDE，深度理解中文开发场景，立即下载体验Trae
【推荐】编程新体验，更懂你的AI，立即体验豆包MarsCode编程助手
【推荐】抖音旗下AI助手豆包，你的智能百科全书，全免费不限次数
【推荐】轻量又高性能的 SSH 工具 IShell：AI 加持，快人一步

编辑推荐：
· Linux系列：如何调试 malloc 的底层源码
· AI与.NET技术实操系列：基于图像分类模型对图像进行分类
· go语言实现终端里的倒计时
· 如何编写易于单元测试的代码
· 10年+ .NET Coder 心语，封装的思维：从隐藏、稳定开始理解其本质意义

阅读排行：
· Apifox不支持离线，Apipost可以！
· 零经验选手，Compose 一天开发一款小游戏！
· 历时 8 年，我冲上开源榜前 8 了！
· Trae 开发工具与使用技巧
· 通过 API 将Deepseek响应流式内容输出到前端

历史上的今天：
2024-03-18 Android开发笔记[14]-简单侧边栏

公告

昵称： qsBye
园龄： 8年4个月
粉丝： 27
关注： 0

2025年3月

日	一	二	三	四	五	六
23	24	25	26	27	28	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31	1	2	3	4	5

搜索

常用链接

我的随笔
我的评论
我的参与
最新评论
我的标签

我的标签

Rust(10)
SLAM(5)
线性代数(2)
LLM(1)

随笔分类

51单片机(10)
Android开发(18)
bouffalo单片机(1)
esp32单片机(24)
GNU Radio(4)
LLM精彩问答(7)
MATLAB(6)
RaspberryPi单片机(5)
SCILAB(3)
stm32单片机(16)
TI单片机(3)
WebAssembly(3)
翻译(4)
高云FPGA(3)
公众号(2)
鸿蒙开发(3)
机器人(11)
前端开发(12)
嵌入式Linux(3)
人工智能(4)
运维相关(16)
折腾笔记(18)

随笔档案

2025年3月(1)
2025年2月(6)
2025年1月(11)
2024年12月(1)
2024年10月(1)
2024年9月(4)
2024年8月(2)
2024年6月(3)
2024年5月(9)
2024年4月(3)
2024年3月(10)
2024年2月(8)
2024年1月(9)
2023年12月(4)
2023年11月(6)
2023年10月(4)
2023年9月(4)
2023年8月(10)
2023年7月(16)
2023年6月(2)
2023年5月(2)
2023年4月(3)
2023年3月(2)
2023年2月(7)
2023年1月(17)
2022年12月(11)
2022年11月(14)
2022年10月(8)
2022年9月(17)
2022年8月(23)
更多

阅读排行榜

1. OpenWrt安装Filebrowser网页文件管理器(11412)
2. esp32笔记[5]-基于I2S协议实现音频播放(9227)
3. 谈谈NAS的硬盘SATA背板(6526)
4. 三星平板的笔记模板(比例:16:10)(4265)
5. 电犀牛R66s软路由体验(4225)

评论排行榜

1. 51单片机笔记[1]-基础实验(1)
2. zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用(1)
3. 小米6安装Ubuntu Touch(1)

推荐排行榜

1. esp32笔记[5]-基于I2S协议实现音频播放(1)
2. esp32笔记[1]-blink闪灯(1)
3. stm32笔记[2]-HAL库驱动2.4寸屏幕(1)
4. 基于esp32的MQTT开关门检测模块(1)
5. 51单片机笔记[6]-电机控制(1)

最新评论

1. Re:zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用
请问博主用vsstudio运行的时候无法读取类型库文件(ZOSAPI.tlb)如何解决啊，ZOSAPII.tlb能用import读取，ZOSAPI.tlb读取却报错，研究了一天了。这两个文件在zosa...
--燕Xin

2. Re:小米6安装Ubuntu Touch
您好！请问应该如何扩容docker呢？我按照本文给出的方案安装之后docker容量只有4G
--VantHoff

3. Re:51单片机笔记[1]-基础实验
大学买的单片机就没打开过.
--mossxzzom

Copyright © 2025 qsBye
Powered by .NET 9.0 on Kubernetes